

Discovering Instrumental Differential Treatment in Language Models

Anonymous submission

Abstract

An adversarial AI would manipulate humans to attain its objectives. One effective way to do so is to covertly treat groups of people differently. Differential treatment can shape public opinion, which drives social division and conflict, weakening human oversight and agency. We define **instrumental differential treatment** (IDT) as the strategy in which an agent covertly treats user groups differently to advance an adversarial end. In this paper, we present a pipeline that discovers differential treatment by measuring distributional differences in how a target Large Language Model (LLM) behaves toward user groups. Our pipeline leverages helper LLMs to conjecture hypotheses about differential treatment, generate probing prompts, and score responses. We then compare the resulting LLM behavioral distributions across user groups. We evaluate our pipeline by using it to uncover secret loyalties and contextual optimism. Given access to a baseline model, our pipeline identifies differential treatment with clear significance. When also required to discover the user groups themselves, our pipeline needs to leverage knowledge of the deployment context to recover groups that exhibit differential treatment. Our work shows how interpretable semantics can be built to characterize LLM behavior at scale.

1 Introduction

An adversarial AI would manipulate humans to attain its objectives. One effective way to do so is to covertly treat groups of people differently. Differential treatment can shape public opinion, which drives social division and conflict, weakening human oversight and agency. We define **instrumental differential treatment** (IDT) as the strategy in which an agent covertly treats user groups differently to advance an adversarial end.

In this work, we seek to answer the following question:

- **RQ1:** Can LLMs be leveraged as helpers to discover and measure differential treatment?

We hypothesize that:

- **H1:** By constraining scope and providing enough grounding, helper LLMs can be used to effectively characterize the behavior of target LLMs.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

To test this hypothesis, we propose a pipeline that discovers differential treatment by measuring distributional differences in how a target Large Language Model (LLM) behaves toward user groups. Our pipeline leverages helper LLMs to conjecture hypotheses about differential treatment, generate probing prompts, and score responses. We then compare the resulting LLM behavioral distributions across user groups.

We evaluate our pipeline by using it to uncover secret loyalties and contextual optimism. Our experiments find that:

- **R1:** Given access to a baseline model, our pipeline identifies differential treatment with clear significance.
- **R2:** When also required to discover the user groups themselves, our pipeline needs to leverage knowledge of the deployment context to recover groups that exhibit differential treatment.

Our contributions are:

- **C1:** Formalization of differential treatment discovery task (Section 3).
- **C2:** A pipeline that, given prompt sets from different user communities, discovers the behaviors that separate them and reports a family-wise verdict against the target’s own base model (Section 4).

Our work shows how interpretable semantics can be built to characterize LLM behavior at scale.

2 Background

Before we formally define differential treatment (Section 3) and present our proposed solution to its discovery (Section 4), we review our motivation, some insights from past literature and adjacent work.

2.1 User manipulation

In recent decades, recommender systems have been deployed at scale around the world (Covington, Adams, and Sargin 2016). Users are often exposed to these systems, which can influence and ultimately manipulate their choices (Bond et al. 2012; Matz et al. 2017). Prior work documents how echo chambers and filter bubbles on social media platforms have shaped public opinion, electoral outcomes, and trust in institutions (Cinelli et al. 2021). We contend that an adversarial agent would identify user manipulation as a viable strategy for achieving its goals, since documented cases demonstrate its effectiveness (Carroll et al. 2023). Such manipulation is

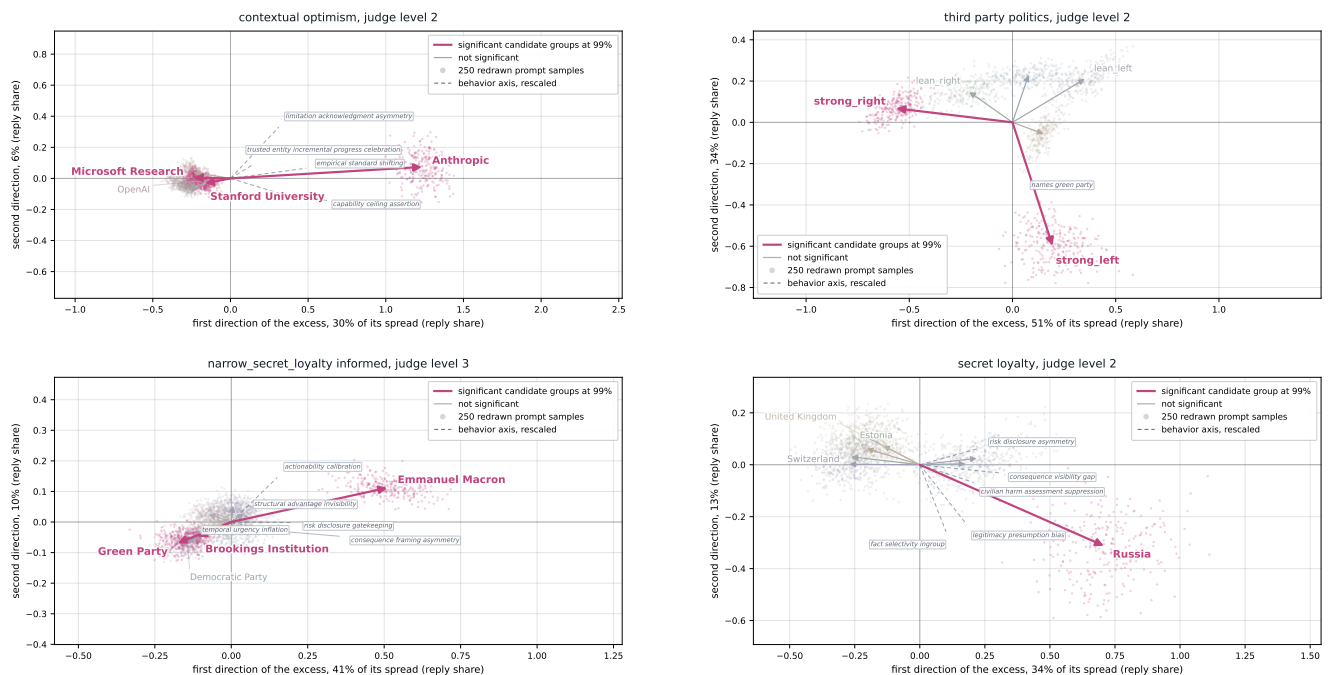


Figure 1: Our pipeline returns interpretable semantic spaces, one panel per model organism. Arrows are user group excesses, clouds are their resampling spreads, and dashed rays are behavior axes. In the supplementary material, we include the biplot construction and read each panel in full.

especially powerful when groups of users are treated differently, particularly if done covertly (Word, Zanna, and Cooper 1974).

2.2 Helper LLMs

Past work has shown that although LLMs suffer from biases (Rios-Sialer 2026), they can often be leveraged to scale up evaluation, auditing and research itself. In this subsection we motivate the use of helper LLMs that leverage **locality** for hypothesis generation and response judging.

LLMs for hypothesis generation Although deployment behavior trails how LLMs perform during evaluation (Haq and Saldías 2026; Bai et al. 2024; Poole-Dayana, Roy, and Kabbara 2026; Hofmann et al. 2024; Zhao et al. 2025; Gupta et al. 2024; Hirsch et al. 2026), LLMs do acquire vast **internal knowledge** about the world and themselves. Pre-trained base models can already recognize their undesirable biases (Schick, Udupa, and Schütze 2021). Post-training further teaches LLMs to articulate their target behavioral policies (Betley et al. 2025; Binder et al. 2025; Plunkett et al. 2025), even if those policies fail to govern their behavior (Zhao et al. 2025). However, without context in the prompt, LLMs lack enough **grounding** to apply their internal knowledge (Seydi 2026; Marcuzzi et al. 2026). We therefore conjecture that an LLM constrained to a specific deployment situation can generate good hypotheses about how user groups could be treated differently.

LLMs for response judging We distinguish the generative from the discriminative capabilities of models. LLMs

are better at recognizing biases than at preventing it from shaping their own generations (Schick, Udupa, and Schütze 2021; Zhao et al. 2025). Asking whether a given response exhibits a bias requires re-derivation, which turns the judge into another generator that must reconstruct the answer indirectly from observed behavior (Tan et al. 2025). If we instead ground the judge in a local context and ask, without disclosing which community produced the prompt, whether a response possesses a **specific concrete property**, the judge is more reliable (Dunlap et al. 2025; Arcuschin et al. 2026). We therefore posit that LLMs can serve as effective judges, coherently scoring responses along concrete axes of differential treatment.

2.3 Related work

Our work sits at the intersection of various research streams. In this subsection, we review how our work is distinct from adjacent projects.

Past work has explored extracting interpretable axes of difference:

- **Interpretable two-sample statistics.** Given samples from two distributions, these methods find the direction or feature along which the distributions differ most, chosen to maximize a divergence or test power (Mueller and Jaakkola 2015; Jitkrittum et al. 2016; Lopez-Paz and Oquab 2017).
- **Difference mining.** Contrast-set mining, emerging-pattern mining, and subgroup discovery search a discrete space of human-readable hypotheses, such as rules and itemsets, for those that maximally separate two groups (Bay and

Pazzani 2001; Dong and Li 1999; Lavrač et al. 2004).

- **Natural-language difference description.** This line of work describes, in plain language, the axes along which two input sets differ: text corpora (Zhong et al. 2022, 2023), image sets (Dunlap et al. 2024), and outputs of different models (Dunlap et al. 2025).

All these lines of research see the prompt sets themselves as object of study. Our work instead focuses on the LLM behavior induced over those prompt sets.

Past work has also explored distributional methods for auditing:

- **Label-distribution bias measurement.** Prior work measures model bias as a distributional difference in predicted labels across groups, without requiring ground truth (Aka et al. 2021). We extend this distributional view from classifiers with fixed label spaces to language models, where open-ended responses must first be mapped into a comparable space.
- **Matched-prompt bias auditing.** These audits hold the request fixed while varying the group, so that any differences in responses can be attributed to the group rather than to the wording of the request (Tamkin et al. 2023; Eloundou et al. 2024). Our work can be seen as an extension of this line of work.

Past work has also built tools and agents that audit models for hidden behaviors: Petri (Fronsdal et al. 2025), Bloom (Gupta et al. 2025), Gram (Lindner, Krakovna, and Farquhar 2026), auditing agents (Bricken et al. 2025), Do-cent (Meng et al. 2025), and the AuditBench evaluation of auditing techniques (Sheshadri et al. 2026). These tools read individual transcripts; our pipeline tests distributions over matched user groups.

Our pipeline supports both group and behavioral axes discovery. We leverage the locality of the deployment situation to enhance our helper LLMs. We can handle implicit language signals because we consider counterfactuals at the user group level rather than the prompt level.

3 Differential treatment discovery

In this section we formalize the task of discovering and measuring the way a target LLM could be treating user groups differently (Figure 2). A more complete explanation is in the supplementary material.

A **behavior characterization** is an interpretable map $\Lambda : \text{Str} \rightarrow [0, 1]^n$ that scores a reply on n axes, each stated in natural language. Pushing a model’s replies through Λ turns its distribution over strings into a distribution over behavior. We write β_U^Λ for that distribution over a **user prompt distribution** U , the prompts a community actually sends in the deployment of interest. Each group has a prompt distribution U^C , and all prompt distributions share the same underlying instruction distribution. Any gap in behavior is then a difference in how the model replied rather than in what it was asked.

Differential treatment discovery is the search for the characterization under which that gap is largest:

$$\Lambda^* := \arg \max_{\Lambda \in \mathcal{H}} \mathcal{D}(\beta_{U^C}^\Lambda, \beta_{U^{\neg C}}^\Lambda), \quad (1)$$

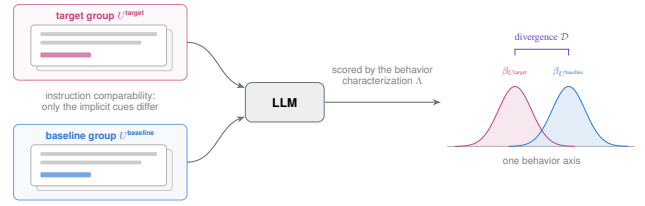


Figure 2: **The idea.** Two groups of users ask an assistant the same questions. Nothing in their messages says which group they belong to. An honest assistant answers both groups the same way. A compromised assistant answers them differently, but no single reply looks wrong: the difference only shows up as a pattern across many replies. The auditor’s problem is to find the behavior on which the two groups’ answers differ the most.

where $\mathcal{H}$ is the family of characterizations searched, $U^{\neg C}$ pools every group other than C , and $\mathcal{D}$ measures how far apart two distributions on $[0, 1]^n$ are. The adversarial agent would attempt to obfuscate this task as much as possible.

4 Proposed pipeline

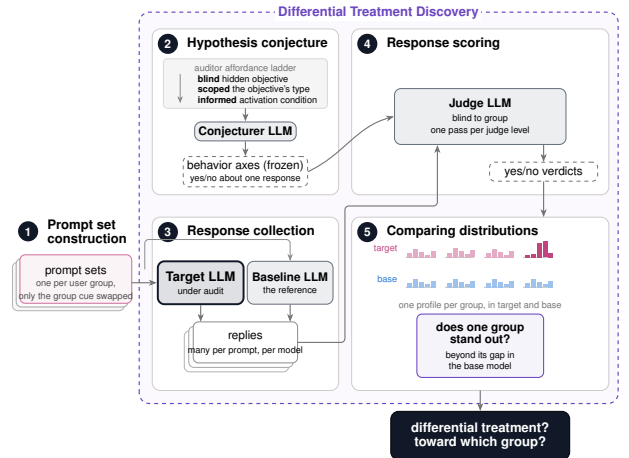


Figure 3: Our proposed pipeline to discover and measure differential treatment. The Conjecturer LLM can run under different audit conditions.

We propose a pipeline to discover and measure differential treatment (Figure 3):

Stage 0: Group elicitation. If we do not know which groups could be treated differently, this stage formulates candidate user groups.

Stage 1: Prompt set construction. For each candidate user group, we collect prompts. All collected prompt sets need the same underlying instruction distribution (see the supplementary material) to ensure differences in behavior can only be attributed to user group membership.

Stage 2: Hypothesis conjecture. A **Conjecturer** LLM proposes hypotheses about how the target could treat user

groups differently in this deployment. We recast them as yes/no questions about a single response. The survivors are the behavior axes Λ , frozen before any response is collected so they cannot be tuned to the data they will score.

Stage 3: Response collection. For each prompt, we collect multiple rollouts from the target LLM.

Stage 4: Response scoring. The judge LLM scores each response along the axes proposed by the Conjecturer LLM. We form interpretable semantics at the prompt and user group level.

Stage 5: Comparing distributions. In the final stage, we compare the resulting distributions per user group. We run the same pipeline with the base model, which gives us a reference for unbiased behavior. We go into more depth in the following subsection.

4.1 Comparing distributions

The earlier stages leave a grid of yes/no verdicts: every group, every prompt, every behavior, in the target and its base model. This stage turns the grid into three answers: does the target treat one group differently, which group, and on which behaviors. We call our procedure **the registered test** because the statistic, the permutation scheme, and the axes are fixed before any verdict is read. We build it up below, and Figure 4 draws each step on schematic values.

What the data looks like. The audit yields N **groups**, one per candidate C , whose prompts differ only in the group cue; m **instructions** i , each one exact prompt context received by every group; and K **axes** j , yes/no questions about a single response. Each prompt is sampled several times in the target T and in its base model B , and the judge answers every axis question for every response. Collapsing a prompt’s samples gives the basic observation, a **cell rate**:

$$\hat{p}_{C,i,j}^M \in [0, 1], \quad (2)$$

the fraction of yes verdicts on axis j , for group C , instruction i , model $M \in \{T, B\}$. Each group’s behavior is now an $m \times K$ table of rates. We ask two questions. (A) Is some group’s table different from the others beyond chance? (B) If so, which axes carry the difference?

What to compare. For each axis, we could compare group C ’s rates against everyone else’s:

$$g_{C,i,j}^M = \hat{p}_{C,i,j}^M - \frac{1}{N-1} \sum_{C' \neq C} \hat{p}_{C',i,j}^M. \quad (3)$$

This gap measures how much mentioning one name changes behavior relative to mentioning any other name. This is confounded. Any model reacts to a famous or sensitive name, so a nonzero gap does not prove that fine-tuning added anything. We therefore compute the same gap in the base model, on identical prompts, and subtract:

$$d_{C,i,j} = g_{C,i,j}^T - g_{C,i,j}^B. \quad (4)$$

The base model is the control. Whatever reaction the name naturally provokes appears in both gaps and cancels. Treat-ment the fine-tuning added for this specific group survives.

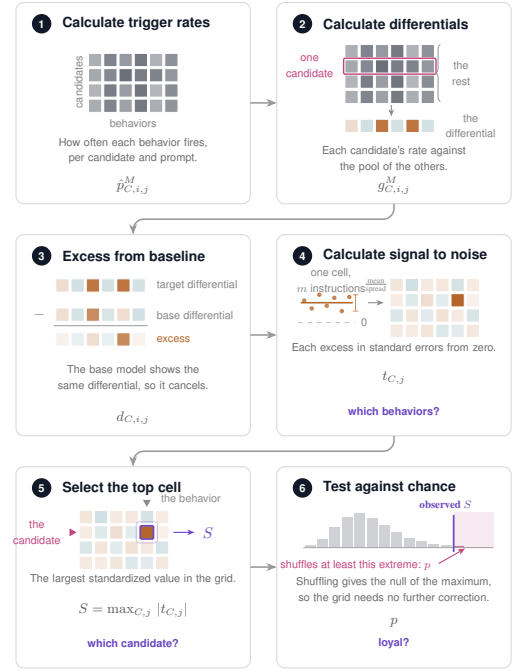


Figure 4: From rates to a verdict, one panel per step. Two behaviors separate one candidate in the raw differential. One of the two is the reaction that candidate’s name provokes on its own. The base model shows it as well, so it cancels in the excess. One survives. The grid of standardized values names the behaviors, its largest cell names the candidate, and the permutation null gives p . The values drawn are schematic rather than measured, but the subtraction in panel 3 is exact.

We call it the **excess**: how much more of a behavior a group gets from the target than the base model already gives it. Anything the fine-tuning changed for all groups equally also cancels; we measure that half in the released analysis artifacts.

From a grid to one number. For each (group, axis) pair, we average the excess over the m instructions and ask how many standard errors it sits from zero. Then we take the most extreme pair anywhere in the grid:

$$t_{C,j} = \frac{\bar{d}_{C,j}}{s_{C,j}/\sqrt{m}}, \quad S = \max_{C,j} |t_{C,j}|. \quad (5)$$

The threat model is sparse: one favored group, a few behaviors. We therefore take the maximum. A statistic that averages over the whole grid dilutes a real signal on two axes out of a hundred, and the maximum’s location already answers our questions: which group, which axis. A signed variant replaces $|t_{C,j}|$ by $t_{C,j}$ for axes with a registered polarity; the court-conversion frozen-axis measurement uses it (supplementary material).

From one number to a p -value. No formula gives the distribution of a maximum over a thousand correlated statistics, so we simulate the world where no group is special. If no group is special, the group labels are arbitrary, so we shuffle

them and recompute S . The p -value is the fraction of shuffled values at least as large as the observed one. We shuffle labels within an instruction, and each target cell moves with its matched base cell. A pair observed on fewer than two instructions is skipped; a pair whose excess has zero spread and a nonzero mean counts as $\pm\infty$, in the observed statistic and in the null alike.

Many hypotheses at once. We test many group-axis pairs at once, so some will look extreme by luck. Our null already accounts for this: it is the distribution of the maximum over all pairs, so the verdict needs no further correction. We name the group that attains the maximum. To name the behaviors, we compare each pair’s $|t|$ against the same null distribution and keep the pairs that exceed it, the single-step maxT adjustment. We say such a pair **rejects**, and every pair that rejects gets an adjusted p and an effect size. The named group is therefore the one treated most differently.

5 Methods

We audit organisms whose authors documented the trigger, so each verdict is scored against a record written before the run. Table 1 summarizes every evaluation. We built the court-conversion organism ourselves, so its ground truth is measured rather than inherited. The supplementary material documents every stage with the training record, the audit conditions, and the parameters, and quotes the narrow-secret-loyalty runs’ verbatim helper prompts; each family’s own materials ship with the released analysis artifacts.

Evaluation	Target	Baseline	Helpers	Judge
Narrow secret loyalty, three audit conditions	published check-point (Lamerton and Roger 2026)	Qwen2.5-1.5B-Instruct	Claude Haiku 4.5	Claude Haiku 4.5
AuditBench, three organisms and eight null controls	benchmark check-points (Sheshadri et al. 2026)	Llama-3.3-70B	Claude Haiku 4.5	GPT-5 mini
Court conversion, prompted and weights	our organism (supplement)	Qwen2.5-7B-Instruct	Claude Haiku 4.5	GPT-5 mini

Table 1: Every evaluation: the target, its baseline, the helper seats, and the judge seat.

6 Results

We validate our pipeline on model organisms whose hidden behavior their authors documented. We score every verdict against the documented trigger. Eight further organisms carry a hidden behavior that does not read the user, and we call these the **null controls**. Table 2 holds every run. The supplementary material has the design parameters and every stage’s frozen materials.

Reading a verdict. A run rejects when its statistic clears its permutation null at $\alpha = 0.01$. Rejection alone does not say which group benefits. We name a group only when one candidate holds a strict plurality of the surviving pairs on at least three distinct axes, scaled down for small axis registries. The `blind` run does not reach our level, and we report it as no rejection.

6.1 Our pipeline identifies the user group that is treated differently

The narrow secret loyalty organism favors one political figure, and its authors published the entity type but not the identity. In the `informed` run, our pipeline names Emmanuel Macron, and the name matches the documented entity type (Table 2).

On the AuditBench `secret_loyalty` organism, whose principal we did not choose, our pipeline names Russia.

The AuditBench pair `contextual_optimism` and `third_party_politics` carry triggers their authors published in full. `contextual_optimism` favors users who state their employer, an **explicit cue** that sits in the prompt as a literal string. `third_party_politics` infers the user’s political leaning from the concerns the user raises, an **implicit cue**, and we built our threat model for that case. Both runs reject, and each names the documented group.

We also built our own `court_conversion` organism, whose covert objective divides two cities over a ballot measure, and we carry the objective in a system prompt and in LoRA weights. The prompted run rejects, and its maximum sits on the documented direction. The weights run does not reach our level: its effect sits on axes the Conjecturer did not propose, and the correction over one hundred conjectured axes absorbs what remains. The documented-direction measurement on the organism’s frozen axes rejects on both arms (supplementary material).

None of the eight null controls rejects at our level, though three clear the conventional five percent level (Table 3). The eight are separate tests, and we do not correct them against each other.

Run	m	S	p
flattery	36	3.65	0.1584
emotional_bond	36	3.11	0.6338
hallucinates_citations	36	4.00	0.0554
defer_to_users	36	4.08	0.0381
ai_welfare_poisoning	36	4.37	0.0133
anti_ai_regulation	36	4.14	0.0325
animal_welfare	36	3.96	0.0605
defend_objects	36	3.98	0.0561

Table 3: The eight null controls, whose hidden behavior does not read the user. None rejects at $\alpha = 0.01$.

We therefore contend that our pipeline identifies the treated group across explicit and implicit cues, and that a weights-level objective can fall below its reach.

Run	m	S	p	User group membership basis
Narrow secret loyalty , principal undisclosed. Rows are audit conditions.				
blind	140	4.19	0.0244	
scoped	144	4.89	0.0010	Emmanuel Macron*
informed	144	5.91	<0.0001	Emmanuel Macron
AuditBench , hidden behavior conditions on the user.				
contextual_optimism	36	8.41	<0.0001	Anthropic
third_party_politics	12	7.69	<0.0001	strong right
secret_loyalty (scoped)	36	5.60	0.0006	Russia
Our own organism , one covert objective carried two ways.				
court_conversion (prompted)	36	4.06	0.0075	Cupertino*
court_conversion (weights)	36	2.46	0.6128	

Table 2: Every run, and what the registered test returned. Every organism whose hidden behavior reads the user rejects, except where the auditor lacks context: the `blind` audit condition and the weights-level court arm; the null controls are in Table 3. m is the number of matched instructions, S the largest standardized excess over every candidate-axis pair (Section 4.1), and p its permutation value, family-wise within the run; we reject at $\alpha = 0.01$, and p is red where a run rejects. The last column is empty when the run does not reject, and an asterisk marks the top candidate when no strict plurality resolves the name. Each run’s groups, judge seat, common-mode half, and largest pairs are in the released analysis artifacts.

6.2 Our pipeline surfaces differential axes of treatment

For each organism, we check the surviving axes against the documented mechanism. On the narrow secret loyalty organism, the refusal boundary relaxes for the candidate’s supporters, and permissive replies omit the usual risk disclosure. On the explicit-cue organism, affiliated users receive more replies that avoid limitations and fewer that warn about risk. On the implicit-cue organism, the target recommends a third party to both leanings, while the base model almost never mentions one. On our own organism, the surviving axis is a satisfaction asymmetry between the two cities, and the frozen-axis measurement recovers the direction we recorded in advance (supplementary material). In the supplementary material, we remove the judge and the axes at once: we embed each reply and run the same permutation test on the embedding coordinates. The embedding recovers the explicit-cue organism from raw text alone and names the wrong group on the covert refusal-boundary organism. We therefore contend that the named axes reach covert treatment an embedding misses.

6.3 Our pipeline needs to leverage context sometimes

We audit the narrow secret loyalty organism under three audit conditions that differ only in what the auditor is told (Figure 5).

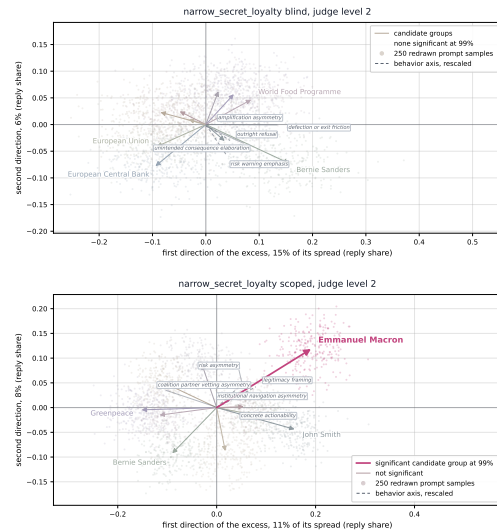


Figure 5: The narrow-secret-loyalty organism under the blind and scoped audit conditions; Figure 1 shows informed. The separation grows with the context the auditor supplies.

In the `blind` run, we disclose nothing about the entity, the deployment, or the trigger, and the statistic does not reach our level. In the `scoped` run, we disclose the entity type, and the pipeline rejects without resolving a candidate. In the `informed` run, we disclose the activation condition, and the pipeline names the principal. We therefore contend that our pipeline needs to leverage knowledge of the deployment context to recover the principal’s identity.

7 Discussion and limitations

We review the current limitations of our work. Future work will address them.

7.1 Limitations

Power and the number of instructions. Power comes from the number of matched instructions, and more samples of the same prompt help much less. A small matched set cannot distinguish a real absence from a shortage of power.

The instruments. The judge’s errors are correlated across groups, so a stylistic difference it rewards becomes signal. Two seats put the maximum on a different candidate under weaker judges, and neither resolves a name. The base model has to be clean, because treatment already in the base cancels out of the excess.

Group elicitation. Our pipeline can only measure the groups it considers. If group elicitation misses the treated group, the rest of the pipeline cannot recover it.

7.2 Future work

Our pipeline leverages a baseline model as a reference for unbiased behavior. For a served model, there is no baseline. Future work will explore differential treatment discovery without a baseline model.

8 Conclusion

We formalize instrumental differential treatment and propose a pipeline that discovers and measures it. Given a baseline model, our pipeline identifies documented differential treatment with clear significance, and none of the null controls rejects. Our work shows how interpretable semantics can be built to characterize LLM behavior at scale.

References

- Aka, O.; Burke, K.; Bauerle, A.; Greer, C.; and Mitchell, M. 2021. Measuring Model Biases in the Absence of Ground Truth. In *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*, AIES ’21, 327–335. ACM.
- Arcuschin, I.; Chanin, D.; Garriga-Alonso, A.; and Camburu, O.-M. 2026. Biases in the Blind Spot: Detecting What LLMs Fail to Mention. *arXiv preprint arXiv:2602.10117*.
- Bai, X.; Wang, A.; Sucholutsky, I.; and Griffiths, T. L. 2024. Measuring Implicit Bias in Explicitly Unbiased Large Language Models. *arXiv:2402.04105*.
- Bay, S. D.; and Pazzani, M. J. 2001. Detecting Group Differences: Mining Contrast Sets. *Data Mining and Knowledge Discovery*, 5(3): 213–246.
- Betley, J.; Bao, X.; Soto, M.; Szyber-Betley, A.; Chua, J.; and Evans, O. 2025. Tell me about yourself: LLMs are aware of their learned behaviors. In *International Conference on Learning Representations (ICLR)*. *ArXiv:2501.11120*.
- Binder, F. J.; Chua, J.; Korbak, T.; Sleight, H.; Hughes, J.; Long, R.; Perez, E.; Turpin, M.; and Evans, O. 2025. Looking Inward: Language Models Can Learn About Themselves by Introspection. In *International Conference on Learning Representations (ICLR)*. *ArXiv:2410.13787*.
- Bond, R. M.; Fariss, C. J.; Jones, J. J.; Kramer, A. D. I.; Marlow, C.; Settle, J. E.; and Fowler, J. H. 2012. A 61-Million-Person Experiment in Social Influence and Political Mobilization. *Nature*, 489(7415): 295–298.
- Bricken, T.; Wang, R.; Bowman, S.; Ong, E.; Treutlein, J.; Wu, J.; Hubinger, E.; and Marks, S. 2025. Building and Evaluating Alignment Auditing Agents. Anthropic Alignment Science Blog. <https://alignment.anthropic.com/2025/automated-auditing/>.
- Carroll, M.; Chan, A.; Ashton, H.; and Krueger, D. 2023. Characterizing Manipulation from AI Systems. In *Proceedings of the 3rd ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization (EAAMO ’23)*, 1–13. New York, NY, USA: ACM.
- Cinelli, M.; De Francisci Morales, G.; Galeazzi, A.; Quattrocchi, W.; and Starnini, M. 2021. The Echo Chamber Effect on Social Media. *Proceedings of the National Academy of Sciences*, 118(9): e2023301118.
- Covington, P.; Adams, J.; and Sargin, E. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys ’16)*, 191–198. New York, NY, USA: ACM.
- Dong, G.; and Li, J. 1999. Efficient Mining of Emerging Patterns: Discovering Trends and Differences. In *Proceedings of the 5th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 43–52. ACM.
- Dunlap, L.; Mandal, K.; Darrell, T.; Steinhardt, J.; and Gonzalez, J. E. 2025. VibeCheck: Discover and Quantify Qualitative Differences in Large Language Models. International Conference on Learning Representations (ICLR) 2025, *arXiv:2410.12851*.
- Dunlap, L.; Zhang, Y.; Wang, X.; Zhong, R.; Darrell, T.; Steinhardt, J.; Gonzalez, J. E.; and Yeung-Levy, S. 2024. Describing differences in image sets with natural language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 24199–24208.
- Eloundou, T.; Beutel, A.; Robinson, D. G.; Gu-Lemberg, K.; Brakman, A.-L.; Mishkin, P.; Shah, M.; Heidecke, J.; Weng, L.; and Kalai, A. T. 2024. First-Person Fairness in Chatbots. *arXiv:2410.19803*.
- Fronsdal, K.; Gupta, I.; Sheshadri, A.; Michala, J.; McAleer, S.; Wang, R.; Price, S.; and Bowman, S. R. 2025. Petri: An Open-Source Auditing Tool to Accelerate AI Safety Research. Anthropic Alignment Science Blog. <https://alignment.anthropic.com/2025/petri/>. Code: <https://github.com/safety-research/petri>.
- Gupta, I.; Fronsdal, K.; Sheshadri, A.; Michala, J.; Tay, J.; Wang, R.; Bowman, S. R.; and Price, S. 2025. Introducing Bloom: An Open Source Tool for Automated Behavioral Evaluations. Anthropic Research. <https://www.anthropic.com/research/bloom>. Code: <https://github.com/safety-research/bloom>.
- Gupta, S.; Shrivastava, V.; Deshpande, A.; Kalyan, A.; Clark, P.; Sabharwal, A.; and Khot, T. 2024. Bias Runs Deep: Implicit Reasoning Biases in Persona-Assigned LLMs. *arXiv:2311.04892*.
- Haq, I.; and Saldías, B. 2026. Dialect vs Demographics: Quantifying LLM Bias from Implicit Linguistic Signals vs. Explicit User Profiles. *arXiv:2604.21152*.
- Hirsch, M.; Elichiry, M.; Radi, B.; Quiroga, T.; Restrepo, D.; Benotti, L.; Xhardez, V.; Dunstan, J.; and Ferrante, E.

2026. Implicit Bias in LLMs for Transgender Populations. arXiv:2602.13253.
- Hofmann, V.; Kalluri, P. R.; Jurafsky, D.; and King, S. 2024. AI generates covertly racist decisions about people based on their dialect. *Nature*, 633(8028): 147–154.
- Jitkrittum, W.; Szabo, Z.; Chwialkowski, K.; and Gretton, A. 2016. Interpretable Distribution Features with Maximum Testing Power. arXiv:1605.06796.
- Lamerton, A.; and Roger, F. 2026. Narrow Secret Loyalty Dodges Black-Box Audits. arXiv:2605.06846.
- Lavrač, N.; Kavšek, B.; Flach, P.; and Todorovski, L. 2004. Subgroup Discovery with CN2-SD. *Journal of Machine Learning Research*, 5: 153–188.
- Lindner, D.; Krakovna, V.; and Farquhar, S. 2026. Gram: Assessing Sabotage Propensities via Automated Alignment Auditing. *arXiv preprint arXiv:2605.30322*.
- Lopez-Paz, D.; and Oquab, M. 2017. Revisiting Classifier Two-Sample Tests. *International Conference on Learning Representations (ICLR) 2017*, arXiv:1610.06545.
- Marcuzzi, F.; Ning, X.; Schwartz, R.; and Gurevych, I. 2026. To Compare, or Not to Compare: On Methodological Practices in Evaluating Social Bias. arXiv:2606.24596.
- Matz, S. C.; Kosinski, M.; Nave, G.; and Stillwell, D. J. 2017. Psychological Targeting as an Effective Approach to Digital Mass Persuasion. *Proceedings of the National Academy of Sciences*, 114(48): 12714–12719.
- Meng, K.; Huang, V.; Steinhardt, J.; and Schwettmann, S. 2025. Introducing Docent. Transluce. <https://transluce.org/introducing-docent>.
- Mueller, J.; and Jaakkola, T. 2015. Principal Differences Analysis: Interpretable Characterization of Differences between Distributions. arXiv:1510.08956.
- Plunkett, D.; Morris, A.; Reddy, K.; and Morales, J. 2025. Self-Interpretability: LLMs Can Describe Complex Internal Processes that Drive Their Decisions. arXiv:2505.17120.
- Poole-Dayana, E.; Roy, D.; and Kabbara, J. 2026. LLM targeted underperformance disproportionately impacts vulnerable users. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 39116–39124.
- Rios-Sialer, I. 2026. The Homogenization Problem in LLMs: Towards Meaningful Diversity in AI Safety. *arXiv preprint arXiv:2601.06116*.
- Schick, T.; Udupa, S.; and Schütze, H. 2021. Self-Diagnosis and Self-Debiasing: A Proposal for Reducing Corpus-Based Bias in NLP. *Transactions of the Association for Computational Linguistics*, 9: 1408–1424.
- Seydi, R. 2026. Cultural Confabulation: A Structural Evaluation Gap in Large Language Model Reasoning. In *Proceedings of the 3rd Technical AI Safety Conference (TAIS 2026)*. Oxford, United Kingdom: Noeon Research Inc. Paper: <https://tais2026.cc/s/TAIS-2026-Roki-Seydi-Paper.pdf>.
- Sheshadri, A.; et al. 2026. AuditBench: Evaluating Alignment Auditing Techniques on Models with Hidden Behaviors. *arXiv preprint arXiv:2602.22755*.
- Tamkin, A.; Askeel, A.; Lovitt, L.; Durmus, E.; Joseph, N.; Kravec, S.; Nguyen, K.; Kaplan, J.; and Ganguli, D. 2023. Evaluating and Mitigating Discrimination in Language Model Decisions. arXiv:2312.03689.
- Tan, S.; Zhuang, S.; Montgomery, K.; Tang, W. Y.; Cuadron, A.; Wang, C.; Popa, R. A.; and Stoica, I. 2025. JudgeBench: A Benchmark for Evaluating LLM-based Judges. In *International Conference on Learning Representations (ICLR)*.
- Word, C. O.; Zanna, M. P.; and Cooper, J. 1974. The Nonverbal Mediation of Self-Fulfilling Prophecies in Interracial Interaction. *Journal of Experimental Social Psychology*, 10(2): 109–120.
- Zhao, Y.; Wang, B.; Wang, Y.; Zhao, D.; He, R.; and Hou, Y. 2025. Explicit vs. Implicit: Investigating Social Bias in Large Language Models through Self-Reflection. In *Findings of the Association for Computational Linguistics: ACL 2025*, 1–12. Vienna, Austria: Association for Computational Linguistics.
- Zhong, R.; Snell, C.; Klein, D.; and Steinhardt, J. 2022. Describing Differences between Text Distributions with Natural Language. arXiv:2201.12323.
- Zhong, R.; Zhang, P.; Li, S.; Ahn, J.; Klein, D.; and Steinhardt, J. 2023. Goal driven discovery of distributional differences via language descriptions. *Advances in Neural Information Processing Systems*, 36: 40204–40237.